

La classe P

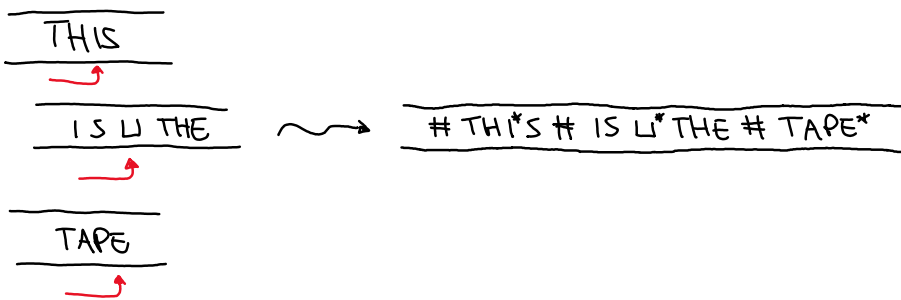
Prima di definirla vediamo degli esempi inerenti alla complessità di tempo

Varianti del modello TM

La TM, M' che può fare $\{L, R, S\}$ si può simulare con la TM, M classica e la complessità di tempo sarà $\leq 2T(m) = O(T(m))$

La TM, M con k nastri, per esempio $k=3$. Si può simulare con la TM, M classica e la complessità di tempo sarà $O(T^2(m))$

Vediamo perché



Immagine

tempo. $O(m)$
 $\# \text{ INPUT } \# L^* \# L^* \# L^*$ $L \quad L$

Simulazione

Per simulare un passo devo scansionare il nastro, ricordandomi i simboli marcati e aggiornare di conseguenza i nastri e proseguire le testine

Inoltre se necessario devo liberare una porzione traslocando tutto a dx di 1 porzione

Quindi al massimo sarà $3T(m) = O(T(m))$

#1 passo $\Rightarrow O(T(m))$

$T(m)$ passi $\Rightarrow O(T^2(m))$

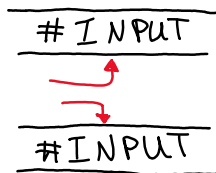
Il totale sarà $O(m) + O(T^2(m)) = O(T^2(m))$

Nella pratica il modello di TM può avere un impatto importante

Ad esempio, consideriamo il linguaggio **palindromes** delle parole palindrome in $\{0,1\}^*$

Se vi ricordate Tale linguaggio è decidibile da una TM con singolo nastro in tempo $O(n^2)$

Con 2 nastri Il tempo $O(n)$



Teorema

Ogni TM a singolo nastro necessita di $\Omega(n^2)$ di tempo per decidere il linguaggio **palindromes**

Per questo corso, queste differenze non contano

Def(DTime)

Sia $t: \mathbb{N} \rightarrow \mathbb{R}^+$, definiamo

$$DTIME(t(n)) = \{ L \mid \exists \text{ una TM che decide il linguaggio } L \text{ in tempo } O(t(n)) \}$$

Ovvero, **Palindromes** $\in DTIME(n^2)$, e $\notin DTIME(n)$

Def(P)

P è la classe dei linguaggi decidibili da una TM in tempo polinomiale

$$P = \bigcup_{k \in \mathbb{N}} DTIME(n^k)$$

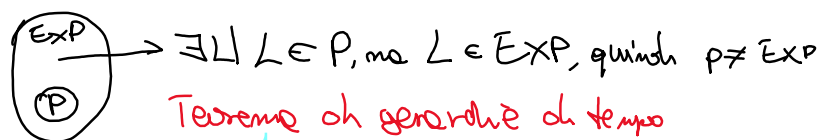
Questa definizione è robusta perché è invariante rispetto al modello di TM

Def(EXP)

EXP è la classe di linguaggi decidibili da una TM in tempo esponenziale.

$$EXP = \bigcup_{k \in \mathbb{N}} DTIME(2^{n^k})$$

Una cosa importante che dimostreremo alla fine è che $P \subsetneq EXP$ fortemente



(P)

Teorema di gerarchia di tempo

$P \subsetneq EXP$, quindi P è contenuta, ma non uguale

La classe EXP è grande, ma ci sono linguaggi banalmente in EXP che in realtà si dimostrano essere in P usando strategie più smart, ovvero semplici

Esempio 1 → grado diretto

$$PATH = \{ \langle G, s, t \rangle \mid \exists s \rightsquigarrow t \text{ in } G \}$$

La codifica di G , $\langle G \rangle$ è una qualunque codifica, del tipo "la matrice di adiacenza"

In questo caso è più comodo contare la complessità in funzione di $n = |V|$ e $m = |E|$, dove $G = (V, E)$

$$s \rightsquigarrow t \quad v_0, v_1, \dots, v_k \mid v_0 = s, v_k = t \text{ e } (v_i, v_{i+1}) \in E$$

La lunghezza del cammino è $\leq n$ e inoltre $\# \text{ di cammini} \leq n^m = 2^{m \log n}$, ovvero $PATH \in EXP$

In realtà $PATH \in P$

marco s

Ripeto

scansiamo tutti gli edge (u, v)

se u è marcato e v no, marco v

mi fermo quando smetto di marcare

Accetto $\Leftrightarrow t$ è marcato

Il tempo è $O(mn) = O(n^3)$

$$m = O(n^2)$$

Quindi $PATH \in P$

Esempio 2

$$2col = \{ \langle G \rangle \mid G \text{ è 2-colorabile} \}$$

Scopo si deve assegnare ai vertici "blu" e "rosso" in modo che i nodi connessi da archi abbiano colori diversi

possibili 2-coloring è $O(2^n)$

$\Rightarrow 2col \in EXP$

Ma in realtà $2col \in P$

Prenolo un vertice e lo coloro di "Blu"

Poi coloro i suoi vertici vicini di "Rosso"

Poi i seguenti vertici vicini al vertice in "Rosso", di "Blu"
e continuo così

Se trovo contraddizioni rifiuto, altrimenti ripeto per ogni componente connessa in G
Se ho colorato tutto, accetto

Esempio 3

3 col? Ovviamente 3 col $\in \text{Exp}$, ma nessuno sa se 3 col $\in \text{P}$

Il miglior algoritmo ha tempo $O(1.333^n)$

Esempio 4

Clique, sono tutti i graph che contengono un triangolo (Δ)

Clique $\in \text{P}$ il tempo per calcolare il $\# \Delta$ è di $O(m^3)$ e controllare se è un Δ costa $O(1)$

Nessuno sa se clique si può decidere in tempo $O(m^2)$

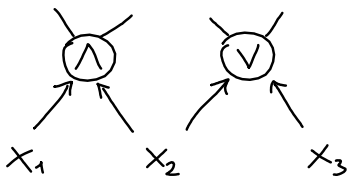
Satisfiability

È il problema fondamentale sulla teoria della complessità. Ci sono tante varianti

Def(Satisfiability)

Un circuito booleano è un graph diretto aciclico con n input $x_1, \dots, x_n$ e 1 output

I suoi vertici vengono chiamati **porte** $\wedge, \vee, \neg$



Se FAN-OUT=1, c'è una formula

Def(Circuit-eval)

$$\text{Circuit-eval} = \{ \langle C, x \rangle \mid C(x) = 1 \}$$

$\langle C \rangle$ è una qualsiasi codifica in funzione di un $\#$ di input, e $\#$ di porte

$$|\langle C \rangle| = O(m \log m)$$

Circuit-eval $\in \text{P}$

Def (Circuit-SAT)

$$\text{Circuit-SAT} = \{ \langle C \rangle \mid \exists x \in \{0,1\}^* \mid C(x) = 1 \}$$

Ovviamente, circuit-sat $\in \text{Exp}$, perché posso deciderlo in tempo $O(\text{poly}(n) 2^n)$

Circuit-SAT $\in P$? Non lo so nessuno

Vedremo che circuit-sat $\in P \Leftrightarrow P = NP$

Varianti

Formula-SAT C è una formula

CNF-SAT C è una CNF, cioè un grande "AND" di clausole in OR di letterali ("V" di letterali)

K-SAT Tutte le clausole hanno K letterali

Esempio 1

3-SAT, ovvero $K=3$ Ovviamente 3-SAT $\in \text{Exp}$, ma 3-SAT $\in P \Leftrightarrow P = NP$

Stessa cosa per CNF-SAT e Formula-SAT

I migliori algoritmi sono

3-SAT $\in \text{DTIME}(1,34^n)$

4-SAT $\in \text{DTIME}(1,5^n \text{poly}(n))$

Teorema (2-SAT)

2-SAT $\in P$

Proof. Sia $\phi(x_1, \dots, x_n)$ la nostra formula 2-SAT, con n variabili ed m clausole. Faremo corrispondere alla formula un grafo

Per ogni clausola $x \vee y$ considero 2 implicazioni logiche equivalenti, " $\bar{x} \rightarrow y$ " e " $\bar{y} \rightarrow x$ "

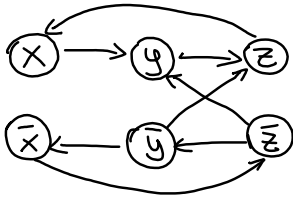
x	y	$x \vee y$	$x \rightarrow y$
0	0	0	1
0	1	1	1
1	0	1	0
1	1	1	1

$$\begin{array}{cc} \bar{x} \rightarrow y & \bar{y} \rightarrow x \\ \vdots & \vdots \\ \vdots & \vdots \end{array}$$

Per ogni $L_1 \vee L_2$ in ϕ , costruisco il graf G con nodi $\{(x_1, \bar{x}_1, \dots, x_n, \bar{x}_n)\}$ aggiungendo anche $\bar{l}_1 \rightarrow l_2$ e $\bar{l}_2 \rightarrow l_1$

Es1

$$\phi(x) = (\bar{x} \vee y) \wedge (\bar{y} \vee z) \wedge (x \vee \bar{z}) \wedge (y \vee \bar{z})$$



Lemma

ϕ è soddisfacibile $\Leftrightarrow$ nessuna componente fortemente connessa di G contiene una variabile e la sua negazione

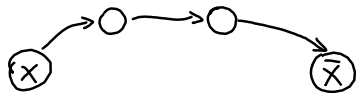
Dim

Fortemente connesso vuol dire che ogni nodo è raggiungibile a partire da qualsiasi altro nodo

($\Rightarrow$)

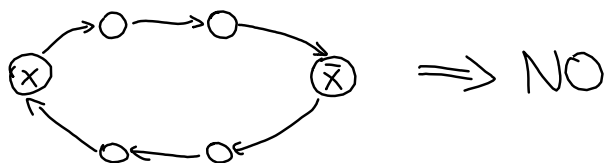
Se ϕ è soddisfacibile e a, b un ora in G . Se $a = T$, allora b deve essere $b = T$, questo perché nella ϕ c'è $(\bar{a} \vee b)$

Se nel graf c'è il cammino $x \rightsquigarrow \bar{x}$



allora è necessario che x sia Falso

Ma allora non può esserci allo stesso tempo il cammino $x \rightsquigarrow \bar{x}$ e $\bar{x} \rightsquigarrow x$



Questo perché, per lo stesso argomento $\bar{x}$ dovrebbe essere falso, ovvero x è vero, quindi $\times$

$\Rightarrow$ ogni componente fortemente connessa non contiene x e $\bar{x}$

($\Leftarrow$)

Assumiamo che nessuna componente connessa contenga x e $\bar{x}$. Ordiniamo le componenti fortemente connesse $C_1, \dots, C_k$

Assumiamo che $\forall x$, scelto $x = \text{True} \Leftrightarrow x$ appare dopo di $\bar{x}$. Altrimenti $x = \text{False}$

Affermazione

Per nessun arco ab di G il vertice $a = \text{True}$ e $b = \text{False}$. Questo implica che ϕ è soddisfacibile

Supponiamo che l'arco ab , con $a = \text{True}$, $b = \text{False}$, e che $a \in C_i$. L'arco ab è presente perché c'è la clausola $\bar{a} \vee b$ che ha pure generato $\bar{b}$

Siccome $a = \text{True}$ e $\bar{a} = \text{False}$, allora $\bar{a}$ appare prima, ovvero $\bar{a} \in C_j$, con $j < i$

Analogamente, siccome $b = \text{False}$ e $\bar{b} = \text{True}$, allora $\bar{b}$ appare dopo C_i , ovvero $\bar{b} \in C_k$, con $k > i$

$\Rightarrow$ l'arco ab contraddice l'ordine delle componenti, perché ci sarebbe un arco tra C_k e C_j , che è dovuto all'arco $\bar{b}\bar{a}$, quindi $\rightarrow \leftarrow$.